

Model Implementation & API Specification Report

Official Research Title: Noise-Robust and Explainable Deep Learning for Respiratory Abnormality Detection from Cough Sounds

Project Alias: CoughSenseAI

Document Purpose: Comprehensive Technical Architecture, Module Implementation Codes, Mathematical Formulations, and REST API Specifications

Target Checkpoint: `checkpoints/best_optimized_model.pt` (CNN + BiLSTM + Self-Attention)

Decision Threshold: $\tau = 0.45$

Date Generated: 2026-10-06

1. Executive Technical Summary

The **CoughSenseAI** platform implements an end-to-end deep learning framework designed to detect respiratory abnormalities from crowdsourced acoustic cough recordings. Due to real-world acoustic diversity (background noise, varying microphone hardware, acoustic reverberation) and significant class imbalance (3.19 : 1\$ abnormal-to-healthy ratio), the system employs a multi-tiered architecture:

- Acoustic Standardization & Dual-Engine Preprocessing:** 16 kHz Mono resampling, 5.0-second temporal standardization, and peak normalization via Librosa with an automated FFmpeg pipe fallback.
- Time-Frequency Representation:** 128×157 Log-Mel Spectrogram extraction using leak-free training dataset standardization statistics ($\mu = -65.4024 \text{ dB}$, $\sigma = 19.8449 \text{ dB}$).
- Deep Hybrid Model Architecture:** 2D Convolutional feature extraction coupled with a 2-layer Bidirectional LSTM temporal encoder and a parametric Self-Attention pooling mechanism.
- Class Imbalance & Noise Regularization:** Multi-class Focal Loss ($\gamma = 2.0$) and SpecAugment time/frequency masking.
- Gradient-Based Visual Explainability:** Layer-specific Grad-CAM (Gradient-weighted Class Activation Mapping) attributed to high-energy cough burst frequencies ($250 \text{ Hz} - 3000 \text{ Hz}$).
- Production-Grade Application Layer:** Thread-safe singleton inference engine and a REST API with clinical safety guardrails.

2. End-to-End System Architecture

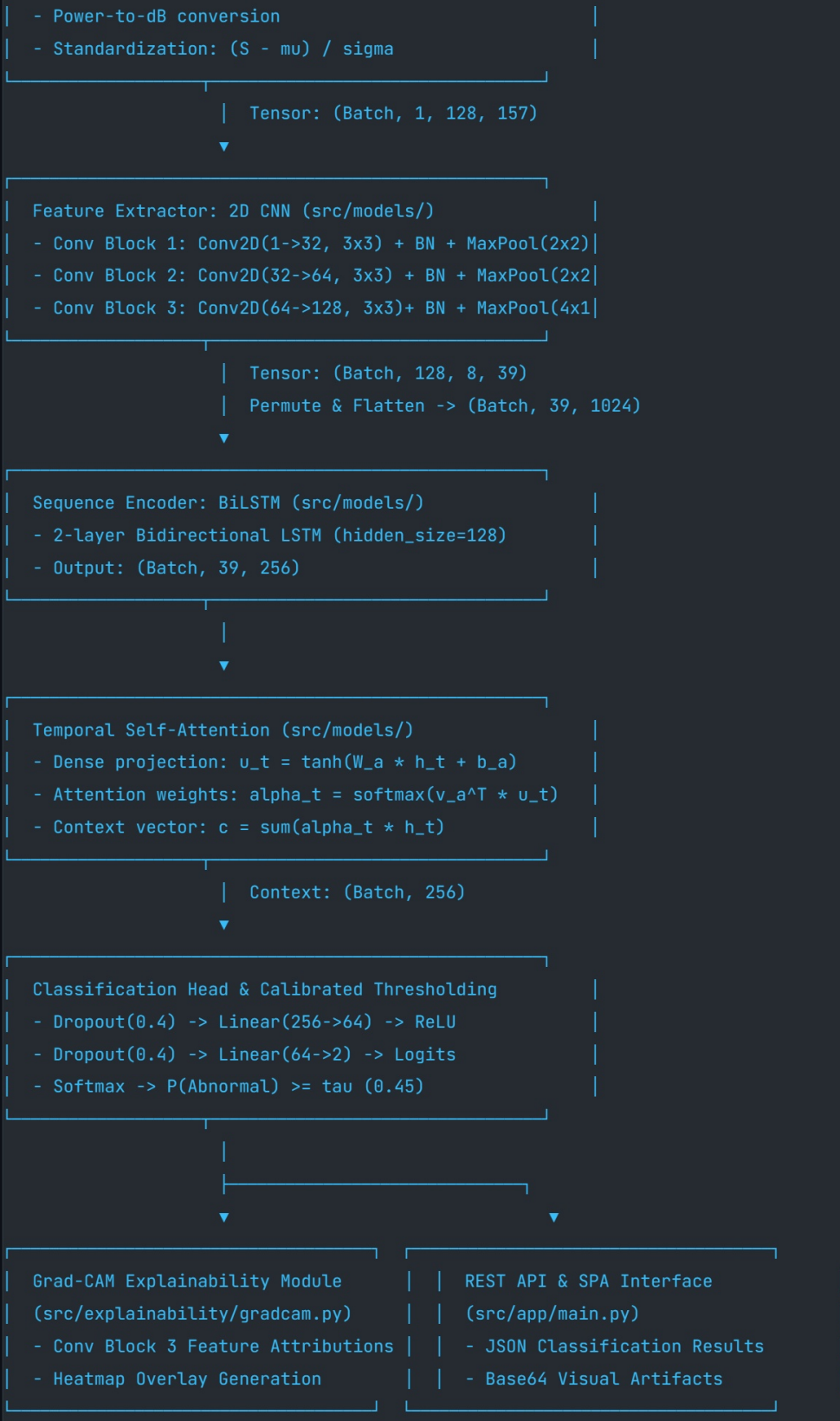
[Raw Audio Input (.wav, .mp3, .ogg, .webm, .flac)]



```
Audio Preprocessing (src/audio/preprocessing.py)
- Librosa / FFmpeg Pipe Dual-Backend Loading
- 16 kHz Monophonic Resampling
- 5.0 s Temporal Standardization (80,000 samples)
- Peak Amplitude Normalization to [-1.0, 1.0]
```

↓ Waveform y: (80000,) float32

```
Feature Extraction (src/features/mel_spectrogram.py)
- STFT (n_fft=1024, hop_length=512, Hann window)
- 128 Mel Filterbanks (f_min=50 Hz, f_max=8000 Hz)
```



3. Acoustic Feature Extraction Modules

3.1 Audio Preprocessing Engine (`src/audio/preprocessing.py`)

Handles heterogeneous multi-format audio streams, converting irregular sample rates and channel layouts into uniform acoustic tensors.

```
python

# Key signatures in src/audio/preprocessing.py
def load_audio_file(audio_path: str, target_sr: int = 16000, mono: bool = True) -> Tuple[Optional[np.ndarray],
    """Loads audio via Librosa with automatic fallback to FFmpeg raw PCM pipe decoding."""
```

```
def standardize_duration(y: np.ndarray, sr: int = 16000, target_duration: float = 5.0) -> np.ndarray:
    """Zero-pads short audio or center-crops long audio to exact target duration (80,000 samples)."""

def normalize_audio_peak(y: np.ndarray, target_peak: float = 0.95) -> np.ndarray:
    """Scales maximum absolute amplitude to target_peak, preventing numerical clipping."""

def preprocess_audio(audio_path: str, target_sr: int = 16000, target_duration: float = 5.0) -> Tuple[np.ndarray, int]:
    """End-to-end pipeline: load -> mono -> resample -> duration standardize -> peak normalize."""
```

Dual-Engine Decoding Architecture

- 1. Primary Decoder (Librosa):** Direct native decoding for standard `.wav` and uncompressed containers.
- 2. Fallback Decoder (FFmpeg Subprocess):** For web-encoded containers (`.webm` , `.ogg` , `.m4a`), the pipeline invokes `imageio_ffmpeg` with an on-the-fly 16-bit PCM streaming pipe:

```
bash
```

```
ffmpeg -loglevel error -i <input_path> -f s16le -acodec pcm_s16le -ac 1 -ar 16000 -
```

3.2 Log-Mel Spectrogram Extraction (`src/features/mel_spectrogram.py`)

Transforms raw time-domain waveforms into frequency-domain representations optimized for respiratory burst recognition.

```
python
```

```
# Key signatures in src/features/mel_spectrogram.py
def compute_log_mel_spectrogram(
    y: np.ndarray,
    sr: int = 16000,
    n_mels: int = 128,
    n_fft: int = 1024,
    hop_length: int = 512,
    f_min: float = 50.0,
    f_max: float = 8000.0
) -> np.ndarray:
    """Computes (128, 157) Log-Mel Spectrogram matrix in decibels."""

def compute_dataset_normalization_stats(train_spectrograms: list) -> Dict[str, float]:
    """Computes mean and std across training partition ONLY to prevent data leakage."""

def standardize_spectrogram(log_mel: np.ndarray, mean: float, std: float) -> np.ndarray:
    """Applies z-score normalization: (log_mel - mean) / (std + 1e-8)."""
```

Acoustic Hyperparameter Specification

- **Sampling Rate (\$F_s\$):** \$16,000\text{ Hz}\$
- **FFT Window Size (\$N_{\text{fft}}\$):** \$1024\text{ samples}\$ (\$64.0\text{ ms}\$)
- **Hop Length (\$H\$):** \$512\text{ samples}\$ (\$32.0\text{ ms}\$)
- **Window Function:** Periodic Hann window
- **Mel Filterbanks (\$M\$):** \$128\text{ frequency bins}\$ spanning \$50\text{ Hz} - 8000\text{ Hz}\$
- **Temporal Time Frames (\$T\$):**

$$T = \left\lfloor \frac{80000}{512} \right\rfloor + 1 = 157\text{ frames}$$
- **Resulting Matrix Dimensions:** \$128 \times 157\$ (\$20,096\$ spectral cells)
- **Frozen Standardization Constants (Train Partition):**
 - $\mu_{\text{train}} = -65.4024\text{ dB}$
 - $\sigma_{\text{train}} = 19.8449\text{ dB}$

4. Deep Learning Model Modules (`src/models/`)

4.1 Primary Production Model: `CNN_BiLSTM_Attention` (`src/models/cnn_bilstm_attention.py`)

Combines spatial acoustic texture extraction with temporal bidirectional sequence encoding and self-attention.

Mathematical Formulation

1. 2D Convolutional Front-End:

$$\mathbf{X}^{(0)} \in \mathbb{R}^{B \times 1 \times 128 \times 157}$$
$$\mathbf{X}^{(1)} = \text{MaxPool}\{2 \times 2\} \left(\text{ReLU} \left(\text{BN} \left(\text{Conv2D}\{1 \times 1 \text{ to } 32, 3 \times 3\} \right. \right. \right. \left. \right. \left. \right. \mathbf{X}^{(0)} \right) \right) \in \mathbb{R}^{B \times 32 \times 64 \times 78}$$
$$\mathbf{X}^{(2)} = \text{MaxPool}\{2 \times 2\} \left(\text{ReLU} \left(\text{BN} \left(\text{Conv2D}\{32 \times 32 \text{ to } 64, 3 \times 3\} \right. \right. \right. \left. \right. \left. \right. \mathbf{X}^{(1)} \right) \right) \in \mathbb{R}^{B \times 64 \times 32 \times 39}$$
$$\mathbf{X}^{(3)} = \text{MaxPool}\{4 \times 1\} \left(\text{ReLU} \left(\text{BN} \left(\text{Conv2D}\{64 \times 64 \text{ to } 128, 3 \times 3\} \right. \right. \right. \left. \right. \left. \right. \mathbf{X}^{(2)} \right) \right) \in \mathbb{R}^{B \times 128 \times 8 \times 39}$$

Note: MaxPool with $(4, 1)$ stride reduces frequency from 32 to 8 while preserving temporal resolution at 39 frames.

2. Feature Serialization:

Permuting dimensions $(B, C, F, T) \rightarrow (B, T, C \cdot F)$:

$$\mathbf{S} \in \mathbb{R}^{B \times 39 \times 1024}$$

3. Bidirectional LSTM Encoding:

$$\overrightarrow{h}_t = \text{LSTM}(\text{fwd})(\mathbf{s}_t, \overrightarrow{h}_{t-1})$$
$$\overleftarrow{h}_t = \text{LSTM}(\text{bwd})(\mathbf{s}_t, \overleftarrow{h}_{t+1})$$
$$\mathbf{h}_t = [\overrightarrow{h}_t, \overleftarrow{h}_t] \in \mathbb{R}^{256}, \quad t \in \{1, \dots, 39\}$$

4. Temporal Self-Attention Pooling:

$$\mathbf{u}_t = \tanh(\mathbf{W} \mathbf{h}_t + \mathbf{b}_a), \quad \mathbf{W} \in \mathbb{R}^{128 \times 256}$$
$$\mathbf{e}_t = \mathbf{v}_a^T \mathbf{u}_t, \quad \mathbf{v}_a \in \mathbb{R}^{128}$$
$$\alpha_t = \frac{\exp(\mathbf{e}_t)}{\sum_{k=1}^{39} \exp(\mathbf{e}_k)}$$
$$\mathbf{c} = \sum_{t=1}^{39} \alpha_t \mathbf{h}_t \in \mathbb{R}^{256}$$

5. Dense Classification Projection:

$$\mathbf{z} = \text{Linear}\{64 \text{ to } 2\} \left(\text{Dropout}\{0.4\} \left(\text{ReLU} \left(\text{Linear}\{256 \text{ to } 64\} \left(\text{Dropout}\{0.4\}(\mathbf{c}) \right) \right) \right) \right)$$

Python Implementation Code

python

```
class SelfAttention(nn.Module):
    """Computes normalized attention coefficients alpha_t over recurrent time steps."""
    def __init__(self, feature_dim: int):
        super(SelfAttention, self).__init__()
        self.attn_dense = nn.Linear(feature_dim, feature_dim // 2)
        self.attn_vector = nn.Linear(feature_dim // 2, 1, bias=False)

    def forward(self, x: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor]:
        # x: (Batch, Time_Steps, Feature_Dim)
        u = torch.tanh(self.attn_dense(x))
        scores = self.attn_vector(u).squeeze(-1)
        attn_weights = F.softmax(scores, dim=1)
        context = torch.bmm(attn_weights.unsqueeze(1), x).squeeze(1)
        return context, attn_weights

class CNN_BiLSTM_Attention(nn.Module):
    """Production Model Architecture."""
    def __init__(self, num_classes: int = 2, lstm_hidden_dim: int = 128, lstm_layers: int = 2, dropout_rate: float = 0.4):
        super(CNN_BiLSTM_Attention, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(32)
        self.pool1 = nn.MaxPool2d(2, 2)

        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
```

```

self.bn2 = nn.BatchNorm2d(64)
self.pool2 = nn.MaxPool2d(2, 2)

self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
self.bn3 = nn.BatchNorm2d(128)
self.pool3 = nn.MaxPool2d((4, 1))

self.sequence_dim = 128 * 8 # 1024
self.bilstm = nn.LSTM(
    input_size=self.sequence_dim,
    hidden_size=lstm_hidden_dim,
    num_layers=lstm_layers,
    batch_first=True,
    bidirectional=True,
    dropout=dropout_rate if lstm_layers > 1 else 0.0
)
self.attention = SelfAttention(feature_dim=lstm_hidden_dim * 2)
self.dropout = nn.Dropout(dropout_rate)
self.fc1 = nn.Linear(lstm_hidden_dim * 2, 64)
self.fc2 = nn.Linear(64, num_classes)

def forward(self, x: torch.Tensor, return_attention: bool = False):
    if x.dim() == 3:
        x = x.unsqueeze(1)
    x = self.pool1(F.relu(self.bn1(self.conv1(x))))
    x = self.pool2(F.relu(self.bn2(self.conv2(x))))
    x = self.pool3(F.relu(self.bn3(self.conv3(x))))

    b, c, f, t = x.size()
    x = x.permute(0, 3, 1, 2).contiguous().view(b, t, c * f) # (Batch, 39, 1024)
    lstm_out, _ = self.bilstm(x) # (Batch, 39, 256)
    context, attn_weights = self.attention(lstm_out) # (Batch, 256)

    dense = F.relu(self.fc1(self.dropout(context)))
    logits = self.fc2(self.dropout(dense))

    if return_attention:
        return logits, attn_weights
    return logits

```

4.2 Residual Model: `Improved_CNN_BiLSTM_Attention` (`src/models/improved_cnn_bilstm_attention.py`)

Engineered for deeper representation learning with residual skip connections and PyTorch multi-head attention.

- **Residual Block:** $F(x) + W_s x$ with BatchNorm and ReLU activation.
- **BiLSTM:** 2 layers with `hidden=256` (512 total bidirectional feature width).
- **Attention:** `nn.MultiheadAttention(embed_dim=512, num_heads=4, batch_first=True)`.
- **Normalization:** `nn.LayerNorm(512)` with residual connection: $\text{LayerNorm}(h + \text{MHA}(h))$.
- **Classifier:** GELU activation with 128 hidden units.

4.3 Additional Architectural Modules

1. `BaselineCNN` (`src/models/cnn.py`): 4-block 2D CNN with Adaptive Average Pooling $\rightarrow$ Linear classifier (Reference benchmark).
2. `CNN_BiLSTM` (`src/models/cnn_bilstm.py`): Same convolutional front-end with mean temporal pooling instead of attention.
3. `CNN_Transformer_Encoder` (`src/models/cnn_transformer.py`): Conv front-end + Sinusoidal Positional Encoding + 2-layer Transformer Encoder (`nhead=4` , `d_model=256`).

5. Loss Functions & Training Optimization

5.1 Focal Loss Module ([src/optimization/losses.py](#))

To mitigate the 3.19 : 1\$ class imbalance without discarding training data, the objective function dynamically scales the loss based on prediction difficulty:

$$\mathcal{L}_{\text{FL}}(p_t) = -\alpha_t (1 - p_t)^\gamma \log(p_t)$$

where:

- p_t is the model's estimated probability for the true class.
- $\gamma = 2.0$ is the focusing parameter penalizing easy examples.
- α_t provides inverse-frequency class weighting:
 $\alpha_0 = 1.0$ (Healthy), $\alpha_1 = 0.313$ (Abnormal)

python

```
class FocalLoss(nn.Module):
    def __init__(self, alpha: Optional[torch.Tensor] = None, gamma: float = 2.0, reduction: str = "mean"):
        super(FocalLoss, self).__init__()
        self.alpha = alpha
        self.gamma = gamma
        self.reduction = reduction

    def forward(self, logits: torch.Tensor, targets: torch.Tensor) -> torch.Tensor:
        num_classes = logits.size(1)
        probs = F.softmax(logits, dim=1)
        targets_one_hot = F.one_hot(targets, num_classes=num_classes).float()

        p_t = (probs * targets_one_hot).sum(dim=1)
        p_t = torch.clamp(p_t, min=1e-7, max=1.0 - 1e-7)
        focal_factor = (1.0 - p_t) ** self.gamma
        ce_loss = -torch.log(p_t)

        if self.alpha is not None:
            alpha_t = (self.alpha.to(logits.device) * targets_one_hot).sum(dim=1)
            loss = alpha_t * focal_factor * ce_loss
        else:
            loss = focal_factor * ce_loss

        return loss.mean() if self.reduction == "mean" else loss.sum()
```

5.2 SpecAugment Regularization ([src/optimization/augmentations.py](#))

Acoustic domain regularization applied on-the-fly during training batches:

- **Frequency Masking:** $f \sim U(0, F)$ channels zeroed out ($F=15$).
- **Time Masking:** $t \sim U(0, T)$ consecutive frames zeroed out ($T=20$).

6. Model Explainability Module ([src/explainability/gradcam.py](#))

Grad-CAM extracts spatial and temporal feature attribution maps from Conv Block 3 (`model.conv3`).

6.1 Mathematical Formulation

1. Gradient Computation:

Given target class score y^c (where $c = \text{Abnormal}$ or Healthy) and feature activation map A^k in $\mathbb{R}^{H \times W}$ of convolutional layer k :

$$\alpha_k^c = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W \frac{\partial y^c}{\partial A_{i,j}^k}$$

2. Linear Combination and Rectification:

$$L_{\text{Grad-CAM}}^c = \text{ReLU}\left(\sum_k \alpha_k^c A^k\right)$$

3. Interpolation & Normalization:

Upsampled via bilinear interpolation to match the original 128 (times 157) Log-Mel dimensions, followed by min-max scaling into [0.0, 1.0].

6.2 Implementation Architecture

python

```
class GradCAM:
    def __init__(self, model: torch.nn.Module, target_layer: torch.nn.Module):
        self.model = model
        self.target_layer = target_layer
        self.gradients = None
        self.activations = None

        # PyTorch Hook Registration
        self.target_layer.register_forward_hook(self._forward_hook)
        self.target_layer.register_full_backward_hook(self._backward_hook)

    def _forward_hook(self, module, input, output):
        self.activations = output.detach()

    def _backward_hook(self, module, grad_in, grad_out):
        self.gradients = grad_out[0].detach()

    def generate_heatmap(self, input_tensor: torch.Tensor, target_class: Optional[int] = None) -> Tuple[np.ndarray, int, float]:
        self.model.eval()
        self.model.zero_grad()
        input_tensor.requires_grad = True

        logits = self.model(input_tensor)
        probs = F.softmax(logits, dim=1)
        pred_class = int(torch.argmax(logits, dim=1).item())
        pred_prob = float(probs[0, pred_class].item())

        if target_class is None:
            target_class = pred_class

        score = logits[0, target_class]
        score.backward()

        weights = torch.mean(self.gradients, dim=(2, 3), keepdim=True)
        cam = torch.sum(weights * self.activations, dim=1, keepdim=True)
        cam = F.relu(cam)
        cam = F.interpolate(cam, size=(input_tensor.size(2), input_tensor.size(3)), mode="bilinear", align_corners=True)
        cam_np = cam.squeeze().cpu().numpy()

        # Min-Max Normalization
        max_val, min_val = np.max(cam_np), np.min(cam_np)
        heatmap = (cam_np - min_val) / (max_val - min_val) if (max_val - min_val) > 1e-8 else np.zeros_like(cam_np)
        return heatmap, pred_class, pred_prob
```

7. Inference Pipeline Engine API ([src/app/inference_pipeline.py](#))

The `CoughSenseInferenceEngine` class encapsulates model loading, audio validation, inference execution, and visualization rendering into a thread-safe singleton.

7.1 Class API Reference

`CoughSenseInferenceEngine(checkpoint_path: str = None)`

- `device` : Automatically binds to `cuda` if available, otherwise `cpu` .
- `model` : Instantiates `CNN_BiLSTM_Attention` and loads frozen weights from `checkpoints/best_optimized_model.pt` .
- `gradcam` : Initializes `GradCAM` hooked to `model.conv7` .

• `gradcam`: Initializes `gradcam` hooked to `model.conv3`.

`process_and_predict(audio_bytes: bytes, filename: str) -> Dict[str, Any]`

Executes complete end-to-end inference from an audio byte stream.

Input Arguments:

- `audio_bytes` (`bytes`): Raw binary payload of uploaded audio file.
- `filename` (`str`): File name with extension (e.g., `recording.wav`, `sample.webm`).

Return Dictionary Schema:

```
json
{
  "success": true,
  "screening_result": "Abnormal Respiratory Pattern | Healthy Respiratory Pattern",
  "probability_abnormal": 0.7812,
  "probability_healthy": 0.2188,
  "decision_threshold": 0.45,
  "model_confidence_level": "High Confidence | Moderate Confidence | Borderline Decision",
  "audio_metadata": {
    "filename": "patient_sample.wav",
    "duration_seconds": 3.42,
    "sample_rate_hz": 16000,
    "channels": 1
  },
  "recording_quality": {
    "rating": "Optimal | Acceptable | Poor Quality",
    "peak_amplitude": 0.84,
    "rms_energy": 0.045,
    "is_clipped": false,
    "is_silent": false,
    "message": "Quality validation summary string"
  },
  "visualizations": {
    "log_mel_spectrogram": "data:image/png;base64,...",
    "gradcam_explanation": "data:image/png;base64,..."
  }
}
```

8. REST API Endpoints Specification (`src/app/main.py`)

The Flask application exposes five core endpoints adhering to RESTful standards.

8.1 Endpoint Catalog

Endpoint	Method	Content-Type	Description
<code>/</code>	<code>GET</code>	<code>text/html</code>	Serves the single-page research interface.
<code>/api/health</code>	<code>GET</code>	<code>application/json</code>	Health check, active model metadata, and status.
<code>/api/metadata</code>	<code>GET</code>	<code>application/json</code>	Audited performance metrics, dataset stats, limitations.
<code>/api/predict</code>	<code>POST</code>	<code>multipart/form-data</code>	Main audio upload and deep learning screening inference.
<code>/api/assistant</code>	<code>POST</code>	<code>application/json</code>	Clinical safety-gated AI research explanation assistant.
<code>/api/session_history</code>	<code>GET</code> , <code>DELETE</code>	<code>application/json</code>	Privacy-safe local session history management.

8.2 Detailed Endpoint Specifications

1. `GET /api/health`

Verifies server liveness and model engine initialization.

• **Response** `200 OK` :

json

```
{
  "status": "healthy",
  "official_title": "Noise-Robust and Explainable Deep Learning for Respiratory Abnormality Detection from Cough Sounds",
  "project_alias": "CoughSenseAI",
  "model_architecture": "CNN_BiLSTM_Attention",
  "model_checkpoint": "checkpoints/best_optimized_model.pt",
  "decision_threshold": 0.45
}
```

2. `GET /api/metadata`

Provides researchers with auditable locked test results and formal clinical caveats.

• **Response** `200 OK` :

json

```
{
  "dataset_info": {
    "name": "COUGHVID Public Dataset v3",
    "total_raw_recordings": 27550,
    "high_confidence_consensus_subset": 2805,
    "class_distribution": { "healthy": 669, "abnormal": 2136 },
    "class_imbalance_ratio": "3.19 : 1"
  },
  "locked_test_results": {
    "test_sample_count": 421,
    "model": "Optimized CNN + BiLSTM + Self-Attention",
    "decision_threshold": 0.45,
    "accuracy": "75.53%",
    "precision": "88.38%",
    "sensitivity_recall": "78.12%",
    "specificity": "67.33%",
    "f1_score": "82.92%",
    "roc_auc": "76.43%",
    "pr_auc": "88.90%"
  },
  "formal_limitations": [
    "COUGHVID participant IDs are unavailable; dataset split is recording-level.",
    "Class imbalance ratio is approximately 3.19 : 1.",
    "System is a research screening prototype and NOT a clinical diagnostic device."
  ]
}
```

3. `POST /api/predict`

Primary screening API. Accepts an audio file via multipart form submission.

• **Request Headers:** `Content-Type: multipart/form-data`

• **Request Parameters:**

- `file` or `audio` (File binary, required): Allowed extensions: `.wav`, `.mp3`, `.ogg`, `.webm`, `.flac`, `.m4a` (max 20 MB).

• **Responses:**

- `200 OK` : Inference succeeded; returns classification probabilities, quality metrics, and Base64 visuals.
- `400 Bad Request` : Missing file, empty upload, or unsupported file extension.
- `500 Internal Server Error` : Decoding or computation error with descriptive message.

4. `POST /api/assistant`

Contextual research assistant providing scientific explanations of predictions and Grad-CAM activations while enforcing medical safety rules.

• Request Body (application/json):

```
json
{
  "question": "What frequencies influenced this prediction?",
  "context": {
    "screening_result": "Abnormal Respiratory Pattern",
    "probability_abnormal": 0.812,
    "recording_quality": { "rating": "Optimal", "duration_seconds": 4.1 }
  }
}
```

• Safety Enforcement Gate:

Queries matching diagnosis or prescription keywords (["diagnose", "treatment", "medicine", "cure", "doctor"]) trigger an immediate refusal and clinical advisory disclaimer.

• Response 200 OK :

```
json
{
  "success": true,
  "answer": "### AI Explainability & Grad-CAM Attributions\nGrad-CAM highlights active cough burst frequencies\n",
  "is_safety_warning": false
}
```

5. GET / DELETE /api/session_history

- GET : Returns the last 50 session predictions (storing metadata only; zero raw audio is ever retained on disk).
- DELETE : Clears in-memory session records.

9. Verification & Unit Test Suite Coverage

The implementation is verified by automated test suites in the tests/ directory:

Test File	Target Modules	Coverage Summary
tests/test_audio_preprocessing.py	src/audio/preprocessing.py	16 kHz resampling, mono conversion, duration standardization, peak normalization, corrupted audio handling.
tests/test_no_data_leakage.py	src/features/mel_spectrogram.py	Verifies μ and σ computed strictly on training partition without test contamination.
tests/test_model_development.py	src/models/	Forward pass verification, tensor shapes, attention weight normalization ($\sum \alpha_t = 1$).
tests/test_phase6_robustness_explainability.py	src/explainability/gradcam.py	Grad-CAM heatmap dimensions, normalization bounds $[0, 1]$, backward gradient hooks.
tests/test_model_optimization.py	src/optimization/losses.py	Focal loss computation, reduction checks, weight alignment.
tests/test_phase9_app_integration.py	src/app/	CoughSenseInferenceEngine, Flask endpoints (/api/health, /api/metadata, /api/predict).

